

DAY 04 • LAB

Out to the Edge, and **Measure** the World.

DURATION

90 minutes

STRUCTURE

15 steps, 6 parts

WORKING

In pairs

YOU NEED

Yesterday's Worker

Measure your own code from twenty cities. Then decide **where things should run**, and defend it.

You will find out how much of a page load is waiting rather than working, watch a flat latency line tilt into a slope the moment your code needs data, and work out why a car cannot ask the cloud whether to brake.

By the end you will have a chart showing what the edge buys you in milliseconds, and a placement decision for nine real workloads that you can argue for.

PART	STEPS	TIME
Instrument it	1 to 2	12 min
Measure from here	3 to 5	15 min
Measure from everywhere	6 to 8	20 min
Break the illusion	9 to 10	12 min
Where should it run	11 to 14	23 min
Findings	15	8 min

Split the work. One drives the Worker, one drives the measurements. Swap at Part 3.

01

Return data instead of a page

Measuring tools can read JSON. They cannot read your typography.

```
export default {
  async fetch(request) {
    const started = Date.now();
    const cf = request.cf || {};
    const url = new URL(request.url);

    const body = {
      served_by:      cf.colo           || "unknown",
      client_country: cf.country        || "unknown",
      client_city:    cf.city           || "unknown",
      client_network: cf.asOrganization || null,
      tls_version:    cf.tlsVersion     || null,
      http_protocol:  cf.httpProtocol   || null,
      time_utc:       new Date().toISOString(),
      worker_ms:      Date.now() - started,
      path:           url.pathname,
    };

    return new Response(JSON.stringify(body, null, 2), {
      headers: {
        "content-type": "application/json;charset=UTF-8",
        "cache-control": "no-store, no-cache, must-revalidate",
        "access-control-allow-origin": "*",
        "timing-Allow-Origin": "*",
      },
    });
  },
};
```

02

Deploy and check

Confirm the JSON loads and that `served_by` shows a three letter code rather than unknown.

Two headers do quiet work. `no-store` stops anything caching the response, so every measurement is real rather than a copy. `timing-Allow-Origin` lets a browser read the detailed timing.

WATCH THIS FIELD

`worker_ms` is how long your code took to run. Note the value now. You will come back to it in Step 15, and it is the whole week in one number.

03

Open the network panel and reload

In your browser's developer tools, Network tab, click the request.

PHASE	WHAT IS HAPPENING	ROUGHLY
DNS	Name into address	0 to 40 ms, then cached
Connect	The handshake	About your round trip
TLS	Agreeing on encryption	One to two round trips
First byte	The wait until the answer starts	The number that matters
Download	Sending the rest	Tiny. Your response is small

04

Run it ten times, take the median

```
macOS / Linux
curl -o /dev/null -s \
-w "ttfb %{time_starttransfer}s\n" URL

Windows PowerShell (note the .exe, it matters)
curl.exe -o NUL -s \
-w 'ttfb %{time_starttransfer}s\n' URL
```

Use the **median**, not the mean. One slow outlier from a wifi retransmission drags a mean somewhere useless, and networks produce outliers constantly.

Windows, read this. In PowerShell, `curl` is an alias for a completely different command. It will not fail cleanly, it will complain about a parameter. Type `curl.exe` every time.

Served by

Median first byteMS

Fastest / slowest of tenMS

05

Count the round trips, then multiply

Opening a fresh connection to any server costs a fixed number of round trips before a single byte of content arrives.

Look up the name	1 round trip
Open the connection	1 round trip
Agree on encryption	1 to 2 round trips
Ask, and be answered	1 round trip
Before any content appears	about 4 to 5

My round trip to my Worker	MS
× 4 round trips	MS
Tuesday's round trip to a far university	MS
× 4 round trips	MS

Now the important part. A faster internet connection does not reduce the number of round trips, and it does not make light travel faster. Buying more bandwidth changes neither number in that table.

This is why "we upgraded the link and it still feels slow" is one of the most common complaints in the industry, and why the answer is almost never more bandwidth.

06

Run global probes

`globalping.io` , no account. Choose an HTTP test, enter your Worker URL, choose World and about 20 probes.

```
npx globalping http \
  YOUR-WORKER.workers.dev \
  from World
```

Backups: `check-host.net` , or `tools.keycdn.com/performance` .

07

Record at least ten locations

PROBE CITY	COUNTRY	FIRST BYTE, MS	SERVED_BY	PROBE TO LOCATION, KM

Decode the three letter code with the airport table from Tuesday, then estimate the distance from the probe to the location that served it.

08

Plot it twice

Chart one. First byte against the probe's distance **from you**.

There should be almost no relationship. You are not in the path.

Chart two. First byte against the probe's distance **to the location that served it**.

Now there is a relationship, and it roughly matches the physics rule you derived on Tuesday.

THE COMPARISON THAT MATTERS

On Tuesday, a request from here to a self hosted university on the far side of the world was somewhere around **250 to 350 ms**.

Your Worker answers most of the planet in **20 to 80 ms**.

You did not write faster code. You wrote the same code, and it stopped crossing oceans. That is the entire proposition of the edge, in one table.

09

Give your code a reason to leave

Everything so far has been a trick with no origin behind it. Real systems have data, and data lives somewhere specific.

```
if (url.pathname === "/origin") {
  const t = Date.now();
  let status = null;
  try {
    const upstream = await fetch("PUT_THE_SINGLE_REGION_URL_HERE", {
      cf: { cacheTtl: 0, cacheEverything: false },
    });
    status = upstream.status;
  } catch (e) {
    status = "failed: " + e.message;
  }
  return new Response(JSON.stringify({
    served_by:      cf.colo || "unknown",
    origin_status:   status,
    origin_round_trip_ms: Date.now() - t,
  }, null, 2), { headers: {
    "content-type": "application/json",
    "cache-control": "no-store" } });
}
```

10

Measure it globally, side by side

Your instructor will give you a single region URL. One machine, in one named city, with nothing in front of it.

PROBE	TO /	TO /ORIGIN

The flat line becomes a slope. **That slope is the ocean.** You can move code anywhere, but the moment it needs data that lives somewhere specific, geography comes back and charges you.

Be considerate. A handful of requests, not a load test.

STEP 11 • YOU HAVE A FIXED BUDGET OF MILLISECONDS. SPEND IT

A • A WEB PAGE THAT FEELS INSTANT

Budget: 100 ms.

Network there and back

Work out the answer

Draw it on screen

B • A VIDEO CALL THAT FEELS NATURAL

Budget: 150 ms, one way.

Capture and compress

Network

Decompress and play

C • A CAR DECIDING WHETHER TO BRAKE

Travelling at 100 km/h, which is **27.8 metres every second**.

Metres travelled in 100 ms

Metres travelled in 20 ms

Budget left for the network

THEN ANSWER THIS

Compare scenario C's network budget with your **best** measured number from Step 7, which was a round trip to a machine in the same city. **Where must the car's decision be made, and what does that tell you about why phones and cars now run models on the device itself?**

STEP 12 · ON THE DEVICE, AT THE EDGE, OR IN THE CLOUD? AND WHAT DECIDED IT

WORKLOAD	DEVICE	EDGE	CLOUD	WHAT DECIDED IT
Face unlock on a phone				
Choosing what to recommend you watch next				
Traffic light timing at one intersection				
Approving a card payment at the moment of tap				
Training a large model				
Obstacle detection in a moving car				
Reading a CT scan for a diagnosis				
Autocomplete in a search box				
Deciding whether an email is spam				

Only four factors are allowed in the last column: latency, data gravity, privacy or sovereignty, cost. If you cannot name one of those four, you have not decided, you have guessed.

13

Count the countries

List every location code you saw today. Decode them. Count the distinct countries.

Locations seen

Distinct countries

Of those, how many you chose

Buildings you could name

14

Now assume it is health records

The law says records for this country's citizens must be processed inside its borders. Write down **three specific changes** you would make.

Before you write, separate two things that are easy to confuse and are not the same question.

WHERE THE MACHINE PHYSICALLY SITS

WHOSE COURTS CAN COMPEL THE COMPANY
OPERATING IT

A location promise. Usually
purchasable, and usually on the
marketing page

A legal question about the operator's
home country. Rarely on the marketing
page

8 MINUTES · IN WRITING

- 01 Look at `worker_ms`. Compare it to your median first byte. What fraction of what the user experienced was your code actually working, and what was everything else?
- 02 Someone proposes fixing a slow application by buying a faster internet connection. Using your Step 5 numbers, explain in two sentences why that may change nothing.
- 03 In your triage table, which workload was hardest to place, and which of the four factors was pulling against which?
- 04 List at least four things the edge cost you. Look at your code for the ones that are not obvious: a long running job, a large file, a database connection held open, or a debugging session at three in the morning.

PRINT THIS FOR THE WINDOWS USERS IN THE ROOM. EVERYTHING IN THIS LAB, BOTH DIALECTS

WHAT YOU WANT	MACOS / LINUX	WINDOWS POWERSHELL
Time one request	curl -o /dev/null -s -w "... " URL	curl.exe -o NUL -s -w '...' URL
Time it without curl at all	time curl -s URL > /dev/null	Measure-Command { Invoke-WebRequest -UseBasicParsing URL }
Run it ten times	for i in {1..10}; do ... ; done	1..10 ForEach-Object { ... }
Fetch the JSON and read it	curl -s URL	Invoke-RestMethod URL
Discard the output	-o /dev/null	-o NUL
Measure globally	globalping.io in a browser, or <code>npx globalping</code> if you have Node. Identical on both	
Read the timing breakdown	Browser developer tools, Network tab. Identical on both	

The honest advice. Everything measured in this lab can be done from the browser network panel and globalping.io, on any platform, with no terminal at all. The commands are faster once they work. They are not required.

THE TRAP THAT WASTES TEN MINUTES

In Windows PowerShell, `curl` is an **alias for a different command**. Typing `curl -w` does not say command not found. It complains about a parameter, which sends you hunting for a typo that is not there.

Always type `curl.exe`. The `.exe` bypasses the alias and reaches the real program, which has shipped with Windows since 2018.

Use **single quotes** around the format string in PowerShell. Double quotes get rewritten before curl sees them.

SYMPTOM	CAUSE	FIX
Every probe reports the same location	You are measuring a cached copy	Confirm no-store is present and deployed
First byte reported as 0 ms	The tool measured a local cache, not the network	Add a changing query string, such as <code>?t=123</code>
served_by shows unknown	You are on the dashboard preview	Use the deployed workers.dev URL
Results vary 30% between runs	Entirely normal for internet measurement	Use medians, and run each test three times
The probe tool stops responding	Free tiers cap probes per minute	Wait a minute, or reduce the probe count
/origin returns failed	The origin is down or blocking you	Tell your instructor. Record it, it is also a finding

HAND IN

Your code ran in a dozen countries today. You chose **none** of them.

ONE

Measurement table, ten locations or more

TWO

Two charts, and your round trip budget

THREE

The three latency budgets, including the car

FOUR

The triage table, all nine placed

FIVE

The four answers from Step 15